

Memory Management

(1) How virtual memory is organized

(2) Paging $\Rightarrow$ (1) Page Tables

$\rightarrow$ Very large in size

(2) Hierarchical Page Table

Page number	Offset
-------------	--------



1st level page number	2nd level page number	Offset
--------------------------	--------------------------	--------

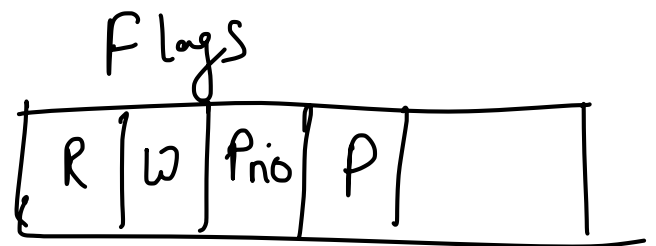
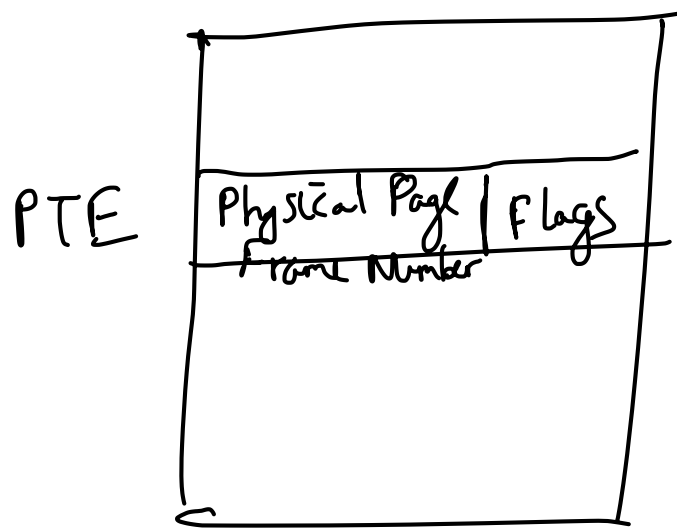
(3) Translation Lookaside Buffer (TLB)

Unsolved Problems $\rightarrow$ (1) Concept of "virtual" memory that is unlimited could not be given to the user.

Swapping: $\rightarrow$ Move pages that are not being used to the secondary storage. This process is called swapping out a page.

Challenges of Swapping: $\rightarrow$ (1) Should not sacrifice performance

(2) Need to keep track of which pages are present in memory.



If not present, then it should be present on the secondary storage. In that case, the page table indicates that no page is present, and a page fault exception is raised. When page fault is raised, the process is moved to the waiting state by the process.

management. You need to identify some page frame to be evicted if the main memory has no free page frames.

What is the best possible page replacement policy?

$\Rightarrow$ "Optimal policy" $\rightarrow$ whichever page will not be used for the longest amount of time in the future should be swapped out.

$\Rightarrow$ "FIFO" $\rightarrow$ Whichever page came first, evict it first. Increasing the size of main memory would increase the number of page faults.

Suppose you have a memory system (M_1) that can store 3 pages. Now, you expand it (M_2) that can store 4 pages. You have the same system (sequence) of accessing the memory.

Suppose $P(M_1)$ and $P(M_2)$ denote the set of pages present.

$$P(M_1) \subseteq P(M_2)$$

There is a sequence of page accesses where the pages present in the first set may not be present in the second set.

⇒ Belady's Anomaly.

(3) LRU (Least Recently Used) ⇒ If we have not used a page for a long time, then it is unlikely we will use it in the near future (temporal locality).

Keeping track of pages used is a challenge. Most systems tend to approximate LRU. (Not Recently Used or NRU).

Can it happen that you are trying to access too many pages at the same time for which we do not have memory space?

→ Thrashing, where you end up spending more amount of time in bringing pages than in actual execution of processes.

→ Can only be handled by killing a few processes.

When do we bring in pages into the main memory?

→ One possibility is to bring in all the pages when a process starts. But a single process can consume a lot of memory.

→ [Demand Paging] → Whenever a request for a page comes through a page fault, only then send it to the main memory.

Most systems try to use a mix of demand paging and proactive paging, to bring in

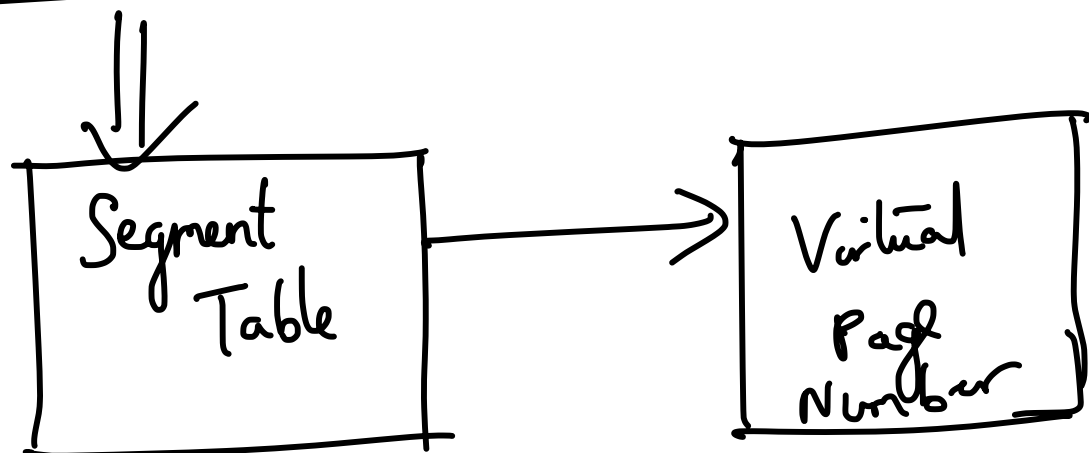
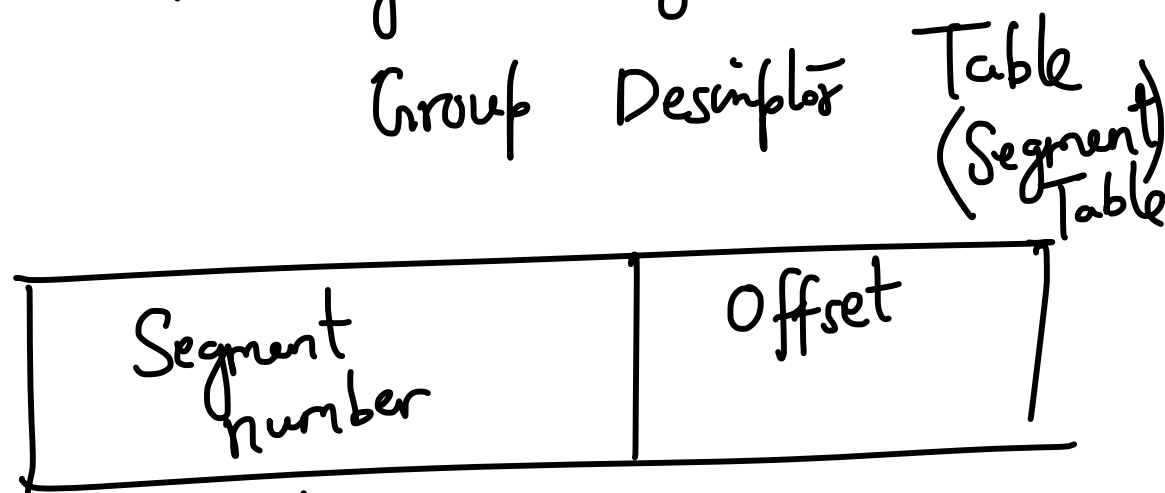
more pages than what caused the page fault.
Usually a system of thresholds is used to control between demand and proactive paging.

What are segments and why do we need them?

⇒ A specific location ^{at} ~~from~~ which a logical group of data is kept/stored → segment

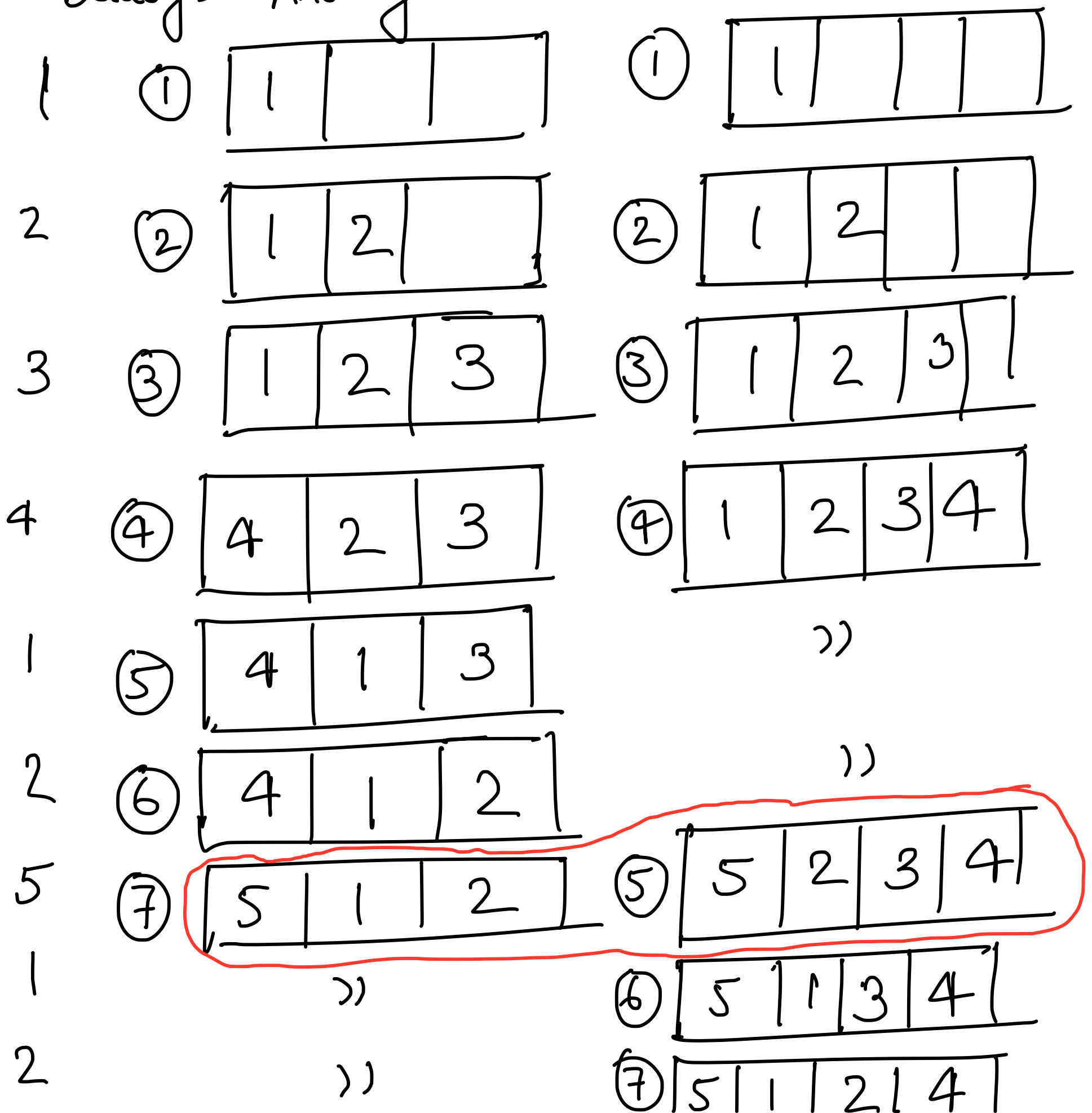
Segments do not have to be equal in size.

→ Segment register



64-bit Intel/x86-64 machines do not support segmentation.

Belady's Anomaly Illustration



3	⑧	5	3	2
---	---	---	---	---

⑧	5	1	2	3
---	---	---	---	---

4	⑨	5	3	4
---	---	---	---	---

⑨	4	1	2	3
---	---	---	---	---

5 1)

⑩	4	5	2	3
---	---	---	---	---

The larger set is not a proper subset
any more, making it possible to
have higher number of page faults.